

Practicas Algoritmia

Santiago Araque

Universidad de Oviedo
22/4/2026

INDICE

Contenido

Practica0	3
Practica 1	8
Practica 1.2	19
Practica 2	23
Practica 3	29
Practica 4 Programación dinámica	30
Practica 5: BackTracking	31
Practica 6. Extra: BackTracking extra	32
Practica 7: Ramifica y poda	33

Practica0

Algoritmia

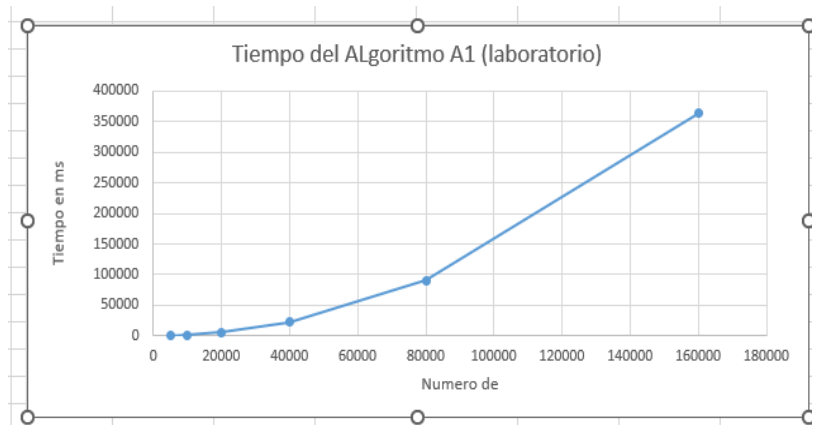
Santiago Araque – UO305904

Toma de tiempos:

1) Factor1: Tamaño del problema

PythonA1.py

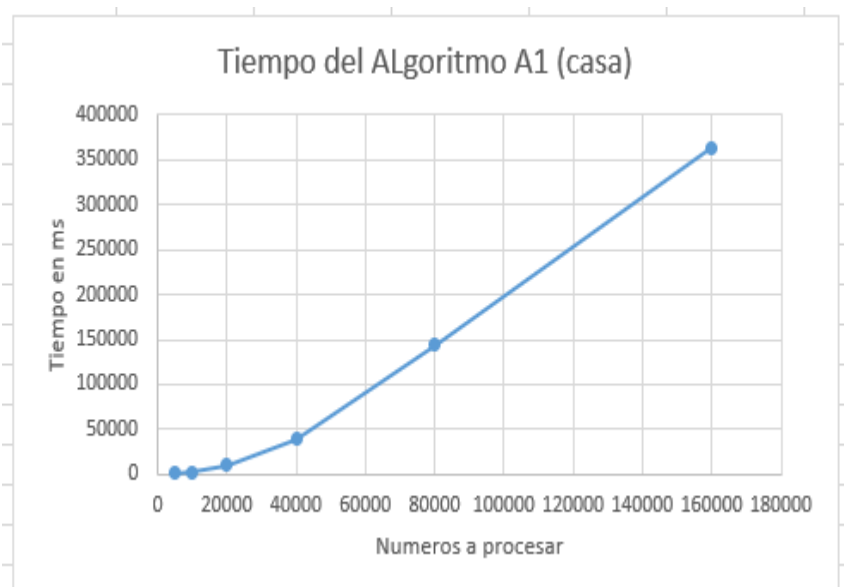
Tiempo Pc del Laboratorio	Tiempo(ms)
5000	329
10000	1348
20000	5452
40000	22374
80000	90855
160000	363815



2) Factor2: Potencia del ordenador

Ordenador de casa: Intel i5-10300H

Tiempo pc casa	Tiempo(ms)
5000	1018
10000	2577
20000	9645
40000	38922
80000	143669
160000	363815



3) Factor: Entorno de implementación:

Java en cmd:

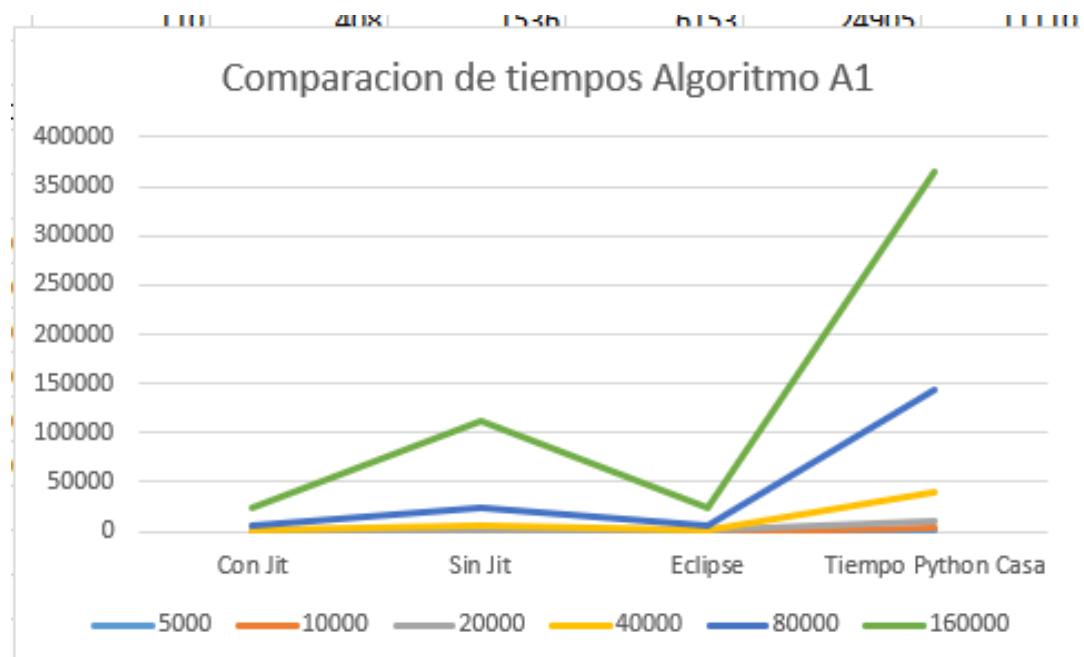
Sin Jit	Tiempo(ms)
5000	110
10000	408
20000	1536
40000	6153
80000	24905
160000	111105

Con Jit	Tiempo(ms)
5000	39
10000	96
20000	375
40000	1552
80000	6035
160000	24278

Java en EclipseIDE:

Eclipse	Tiempo(ms)
5000	42
10000	122
20000	410
40000	1599
80000	6222
160000	24944

Comparación con Python en algoritmo A1:



4) Factor4: Algoritmo Empleado

Pruebas de los Algoritmos en Python

A2	T(ms)	A3	T(ms)	A4	T(ms)
5000	85	5000	36	5000	0
10000	298	10000	135	10000	0
20000	1075	20000	505	20000	0
40000	3933	40000	1996	40000	19
80000	14683	80000	7086	80000	11
160000	54372	160000	26411	160000	23
320000	205403	320000	103410	320000	55
640000	779029	640000	365815	640000	121
				1280000	231
				2560000	520
				5120000	1086
				10240000	2143
				20480000	4420
				40960000	8910
				81920000	18160

Pruebas de los algoritmos en java sin optimización

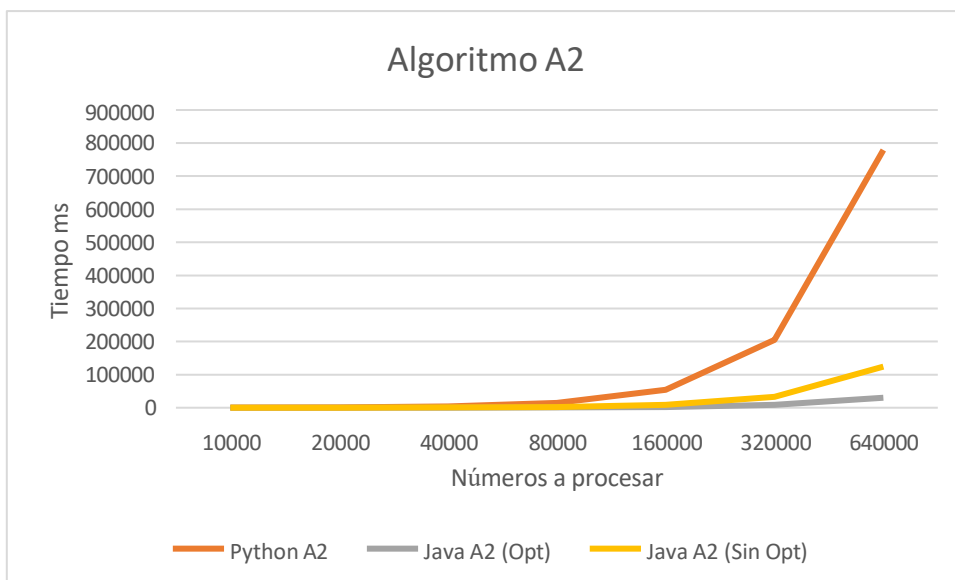
A2	T(ms)	A3	T(ms)	A4	T(ms)
5000	20	5000	14	5000	6
10000	47	10000	30	10000	0
20000	170	20000	111	20000	0
40000	620	40000	429	40000	2
80000	2313	80000	1588	80000	3
160000	8878	160000	5982	160000	6
320000	33164	320000	22490	320000	13
640000	124266	640000	84946	640000	27
				1280000	54
				2560000	112
				5120000	229
				10240000	461
				20480000	934
				40960000	1924

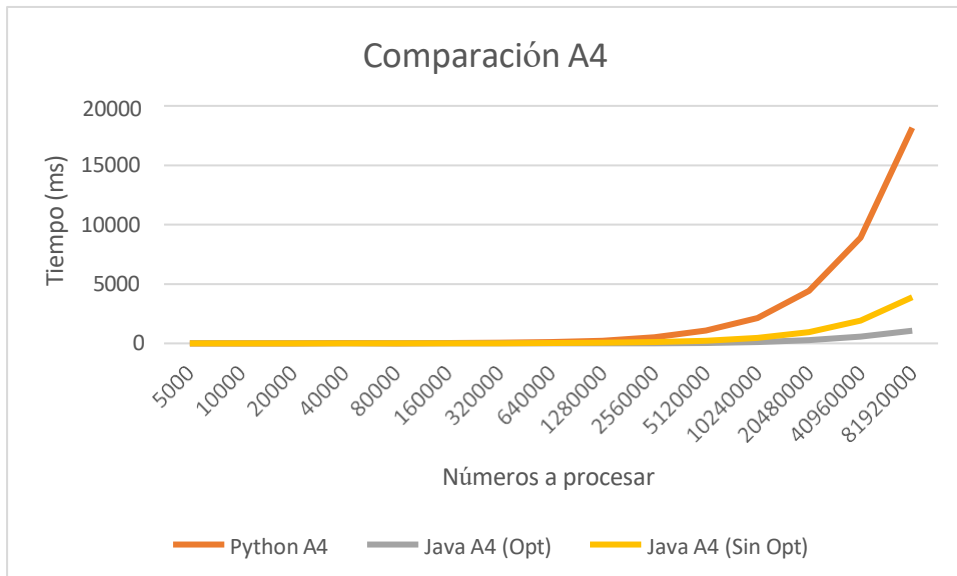
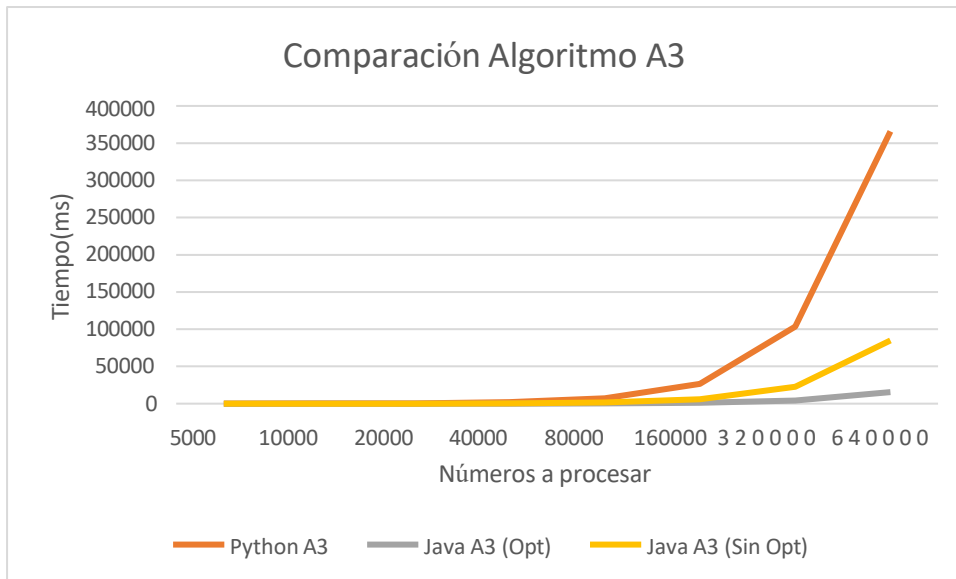
				81920000	3886
--	--	--	--	----------	------

Pruebas de los algoritmos en java con optimización

A2	T(ms)	A3	T(ms)	A4	T(ms)
5000	26	5000	21	5000	16
10000	17	10000	6	10000	0
20000	56	20000	22	20000	1
40000	176	40000	78	40000	2
80000	604	80000	297	80000	3
160000	2217	160000	1095	160000	6
320000	8306	320000	4135	320000	8
640000	30191	640000	15505	640000	11
				1280000	16
				2560000	10
				5120000	27
				10240000	109
				20480000	263
				40960000	561
				81920000	1065

Comparación de los algoritmos A2, A3 y A4





Practica 1

- 1) ¿calcular cuántos años más podremos seguir utilizando esta forma de contar?

Cuando se pinte el valor de la instrucción
System.currentTiemMillis():

1770573700009

Este valor se guarda en un tipo Long (64 bits) que puede almacenar:

18446744073709600000

Por tanto, quedan por almacenar:

18446744073709600000-1770573700009=

18446742303135899991

Este número pasado a años seria:

584942361.21055

2)

Vector2	
Número de elementos del array	Tiempo (ms)
1000000	5
10000000	48
50000000	242
100000000	485
200000000	966

- ¿Por qué a veces el tiempo medido sale 0?

Suele suceder por dos razones principalmente, la primera porque son tiempos muy pequeños, pero normalmente, esto sucede debido a una interferencia con el recolector de basura de la JVM que a veces interrumpe el proceso y lo termina antes de lo pensado.

- ¿A partir de qué tamaño de problema (n) empezamos a obtener tiempos fiables?

A partir de un numero de repeticiones cerca de los 100 millones se obtienen resultados de tiempos alrededor de 600 o 700 milisegundos que pasan a ser resultados más confiables.

- 2) - ¿Qué pasa con el tiempo si el tamaño del problema se multiplica por 2?

Vector4	
Número de repeticiones	10
Números a procesar	Tiempo
10000	1
20000	1
40000	3
80000	6
160000	12
320000	22
640000	45
1280000	97
2560000	181
5120000	389
10240000	728
20480000	1466
40960000	2938
81920000	5883

Vector4	
Número de repeticiones	20
Números a procesar	Tiempo
10000	1
20000	3
40000	5
80000	12
160000	23
320000	46
640000	92
1280000	186
2560000	377
5120000	757
10240000	1491
20480000	2983
40960000	6011
81920000	12329

Se multiplica el tiempo por esa misma constante, es decir, salen tiempos parecidos al doble de que se hubiesen puesto con parámetros potencia de 10

–¿Qué pasa con el tiempo si el tamaño del problema se multiplica por otro k que no sea 2? (Pruebe, por ejemplo, para $k=3$ y $k=4$ y compruebe los tiempos obtenidos.)

Pasa igual que en el apartado anterior, el tiempo se multiplica por esa constante K .

- ¿Razone si los tiempos obtenidos son los que se esperaban de la complejidad lineal $O(n)$?

Se puede evidenciar que los tiempos siguen una complejidad lineal $O(n)$ ya que el tiempo crece de forma proporcional al tamaño de la entrada, donde se evidencia que el tiempo por cada n aumenta casi del doble del apartado anterior

3) El ordenador donde se hicieron las pruebas

Intel i5 10300h

RAM: 8Gb

Parámetro: 1

n	Tsuma	Máximo
10000	0	0
20000	0	1
40000	0	0
80000	1	0
160000	1	1
320000	3	2
640000	5	6
1280000	9	15
2560000	20	22
5120000	37	43
10240000	72	87
20480000	145	174
40960000	290	354
81920000	588	699

Parámetro: 10

n	Tsuma	Tmaximo
10000	1	1
20000	1	1
40000	3	4
80000	6	7
160000	12	13
320000	22	28
640000	45	55
1280000	97	109
2560000	181	220
5120000	389	438
10240000	728	874
20480000	1466	1742
40960000	2938	3476
81920000	5883	6975

Parámetro: 100

n	Tsuma	Tmaximo
10000	7	9
20000	15	18
40000	29	35
80000	57	70
160000	117	136
320000	227	272
640000	454	539
1280000	960	1089
2560000	1829	2171
5120000	3698	4613
10240000	7327	9392
20480000	14679	18380
40960000	30097	36102
81920000	58795	FDT

Parametro:1000

n	Tsuma	Tmaximo
10000	90	87
20000	181	172
40000	356	345
80000	708	679
160000	1413	1366

n	Tsuma	Tmaximo
320000	2840	2722
640000	5680	5454
1280000	11368	10958
2560000	22736	22053
5120000	45536	44189
10240000	FDT	FDT
20480000	FDT	FDT
40960000	FDT	FDT
81920000	FDT	FDT

Parametro:10000

n	Tsuma	Tmaximo
10000	711	875
20000	1434	1810
40000	2842	3616
80000	5661	7103
160000	11328	14177
320000	22764	28467
640000	45645	57163
1280000	FDT	FDT
2560000	FDT	FDT
5120000	FDT	FDT
10240000	FDT	FDT

n	Tsuma	Tmaximo
20480000	FDT	FDT
40960000	FDT	FDT
81920000	FDT	FDT

Parametro:100000

n	Tsuma	Tmaximo
10000	8853	8979
20000	16135	17731
40000	30942	34640
80000	FDT	FDT
160000	FDT	FDT
320000	FDT	FDT
640000	FDT	FDT
1280000	FDT	FDT
2560000	FDT	FDT
5120000	FDT	FDT
10240000	FDT	FDT
20480000	FDT	FDT
40960000	FDT	FDT
81920000	FDT	FDT

Los parámetros 1 000 000 y 10 000 000 serán todos los minutos por tanto son todos fuera de tiempo;

Parámetro: 1

n	TCoincidencias1	TCoincidencias2
10000	690	0
20000	2712	0
40000	10897	0
80000	43260	1
160000	FDT	1
320000	FDT	3
640000	FDT	5
1280000	FDT	12
2560000	FDT	22
5120000	FDT	45
10240000	FDT	90
20480000	FDT	181
40960000	FDT	359
81920000	FDT	738

Parámetro: 10

n	TCoincidencias1	TCoincidencias2
10000	690	1
20000	2705	3
40000	10603	3
80000	41563	7
160000	FDT	13

n	TCoincidencias1	TCoincidencias2
320000	FDT	28
640000	FDT	56
1280000	FDT	113
2560000	FDT	226
5120000	FDT	452
10240000	FDT	926
20480000	FDT	1838
40960000	FDT	3688
81920000	FDT	7380

Parámetro: 100

n	TCoincidencias1	TCoincidencias2
10000	FDT	10
20000	FDT	21
40000	FDT	37
80000	FDT	71
160000	FDT	142
320000	FDT	291
640000	FDT	580
1280000	FDT	1183
2560000	FDT	2339
5120000	FDT	4674
10240000	FDT	9345

n	TCoincidencias1	TCoincidencias2
20480000	FDT	18748
40960000	FDT	37439
81920000	FDT	FDT

Parámetro: 10000

n	TCoincidencias1	TCoincidencias2
10000	FDT	9038
20000	FDT	18107
40000	FDT	35908
80000	FDT	FDT
160000	FDT	FDT
320000	FDT	FDT
640000	FDT	FDT
1280000	FDT	FDT
2560000	FDT	FDT
5120000	FDT	FDT
10240000	FDT	FDT
20480000	FDT	FDT
40960000	FDT	FDT
81920000	FDT	FDT

A partir de aquí todos los tiempos sobrepasan el minuto y por tanto quedan fuera de tiempo

Practica 1.2

Santiago Araque – UO305904

Comparación de los 4 diferentes bucles

n	Tbucle1	Tbucle2	Tbucle3	Tbucle4
100	0	1	1	0
200	0	0	3	6
400	0	2	15	43
800	0	13	67	341
1600	1	59	277	2678
3200	0	244	1207	21305
6400	1	953	4936	241953
12800	2	4357	20508	FDT
25600	3	19314	82771	FDT
51200	7	73633	342431	FDT

Según las complejidades de los 4 bucles:

$$B1=O(n \log)$$

$$B2=B3=O(n^2 \log)$$

$$B4= \frac{2}{3} n^3$$

Se puede deducir que los tiempos esperados son acordes a lo expuesto, ya que el bucle uno se comporta de una manera quasilineal, donde los tiempos aumentan en función de la n, pero el bucle 2 y el 3 se comportan casi parabólicamente, haciendo que sus tiempos aumenten considerablemente con cada ejecución. Finalmente, el 4 bucle, el de mayor complejidad se nota que rápidamente excede la barrera de los 5 minutos a la mitad de la ejecución, debido a una complejidad de grado 3.

Comparación de los bucles 5 6 y 7

Parámetro=10

n	Bucle 5	Bucle 6	Bucle 7
100	71	71	204
200	358	650	2869
400	1760	5824	43361

800	8391	50707	FDT
1600	39880	FDT	FDT
3200	230915	FDT	FDT
6400	FDT	FDT	FDT
12800	FDT	FDT	FDT
25600	FDT	FDT	FDT
51200	FDT	FDT	FDT
102400	FDT	FDT	FDT
204800	FDT	FDT	FDT
409600	FDT	FDT	FDT
819200	FDT	FDT	FDT

Los tiempos mostrados anteriormente comprueban lo esperado, empezando por el bucle 7 que con una complejidad de $O(n^4)$, se ve incrementado sus tiempos en gran medida, donde para n relativamente mediano, sus mediciones se alargan hasta los 10 o 15 minutos, quedando fuera de tiempo.

El bucle 6 con una complejidad $O(n^3 \log n)$, es decir que la función de sus tiempos respecto a n , se comportan casi como una función de grado 3, solamente que, estabilizada por la función logarítmica, pero igual que la anterior con n medianos, su tiempo se dispara muy rápido.

Finalmente, el bucle 5 con complejidad $O(n^2 \log^2 n)$, crece de manera más lenta que los 2 anteriores, sin embargo, tampoco logra completar la tabla, porque, aunque se podría pensar que el logaritmo ralentizará la función cuadrática, este está al cuadrado por tanto el tiempo se verá afectando aumentando un poco más rápido.

Comparación TBucle1 y TBucle2:

n		Tbucle1	Tbucle2	t1/t2
100		0	1	0
200		0	0	0
400		0	2	0
800		0	13	0
1600		1	59	0,01694915
3200		0	244	0
6400		1	953	0,00104932
12800		2	4357	0,00045903
25600		3	19314	0,00015533
51200		7	73633	9,5066E-05

El cociente concuerda con lo esperado porque estamos comparando un algoritmo “lento” frente a uno “rápido”, lo que hace que la división se acerque a cero.

Comparación TBucle2 y TBucle3:

n	Tbucle2	Tbucle3	t3/t2
100	1	1	1
200	0	3	0
400	2	15	7,5
800	13	67	5,15384615
1600	59	277	4,69491525
3200	244	1207	4,94672131
6400	953	4936	5,17943337
12800	4357	20508	4,70690842
25600	19314	82771	4,28554416
51200	73633	342431	4,65050996

A pesar de que ambos bucles poseen la misma complejidad, esta está multiplicada por una constante distinta, lo que hace que, aunque se comporten de manera igual, sus tiempos varían considerablemente. Por tanto, al compararlos obtenemos un cociente que tiende a infinito, aunque no se aprecie lo suficiente.

Comparación Bucle4 java con Bucle4 CPython

repe=1			
n	Bucle Java 4	Bucle Python 4	tBuclePython/tBucleJava
100	0	2	0
200	6	20	3,33333333
400	43	162	3,76744186
800	341	1359	3,98533724
1600	2678	11680	4,36146378
3200	21305	112629	5,28650552
6400	241953	FDT	
12800	FDT	FDT	
25600	FDT	FDT	
51200	FDT	FDT	

La comparación muestra mejor el rendimiento de java en programas con bucles, debido a su compilación que arroja resultados de tiempos más rápidos en contraposición con Python, ya que este último debido a su construcción como lenguaje interpretado, hace que, los bucles y la búsqueda del tipado de las variables conviertan el programa en uno más lento.

Practica 2

Santiago Araque

Algoritmos de sustracción:

Las complejidades fueron obtenidas bajo la fórmula del teorema maestro

$a=1; b=1; k=0$

Complejidad Sustraccion1: $O(n)$

$a=1; b=1; k=1$

Complejidad Sustraccion2: $O(n^2)$

$a=2; b=1; k=0$

Complejidad Sustraccion3: $O(2^n)$

Sustraccion1 y sustracción2, por las pocas medidas que arrojan en su ejecución, se puede evidenciar que los tiempos se comporta tal como establecen las complejidades anteriores. Sin embargo, debido a las múltiples llamadas recursivas que se acumulan en la pila (cada método llamado es una entrada en la misma), el programa se desborda y no termina su ejecución en $n > 8000$.

Para calcular cuánto se demoraría el algoritmo de sustracción 3 para n igual 80, solamente hay que aplicar la formula vista en clase, utilizando un tiempo como referencia. Por eso:

Para $n=30$ sustracción 3 demora 48434 ms

Su complejidad es de $O(2^n)$

Y, además sabemos que un año tiene 31557600000 ms,

Por tanto, operamos

$$T2 = ((2^{80}/2^{30}) * 48434) / 31557600000$$

$$T2 = 6,34E+39 \text{ años}$$

Sustraccion4 Tiempos con 1 repetición

Sustracción 4	
n	
100	4
200	32
400	248
800	2009
1600	15936
3200	136410

Sustraccion5 Tiempos con 1 repetición

Sustracción 5	
n	T(ms)
20	2
22	5
24	16
26	46
28	133
30	401
32	1207
34	3596
36	10833
38	32868
40	119692

Algoritmos de División

Las complejidades fueron obtenidas bajo la fórmula del teorema maestro

$$a=1; b=3; k=1$$

Complejidad Division1: $O(n)$

$a=2$ $b=2$ $K=1$

Complejidad Division2: $O(n\log(n))$

$a=2$; $b=2$; $k=0$

Complejidad Divison3: $O(n)$

Después de la medición de tiempos, se puede concluir que los mismos son acordes a lo expuesto anteriormente. Entrando más en detalle, division1 arroja tiempos que se van multiplicando por 2, asimilando una línea recta como era de esperarse. Division2 es un poco más lento debido a que se hacen más llamadas recursivas, y también, en comparación con el anterior, crece más rápido (porque además de aumentar como n , se multiplica por $\log n$). Por último, Division3, que a pesar de tener la misma complejidad que el primero, muestra tiempos un poco más lentos, esto es debido porque se hacen más llamadas recursivas y además se divide el número de problema en un factor menor haciendo más subproblemas de mayor tamaño que en división1.

Tiempos Division4 con 1 repetición

División 4	
1000	9
2000	35
4000	144
8000	563
16000	2229
32000	9073
64000	36109

Tiempos Division5 con 1 repetición

División 5	
1000	26

2000	104
4000	404
8000	1642
16000	6470
32000	26228
64000	104654

Comparación Vector Suma y Fibonacci

Métodos Vector Suma

Metodo1 - iterativo:

- Complejidad: $O(n)$

Metodo2 – por sustracción:

- Complejidad: $O(n)$

Metodo3 – por división:

- Complejidad: $O(n)$

Vector Suma			
n	Metodo1	Metodo2	Metodo3
40	538	1679	2907

Se puede evidenciar una clara victoria para el algoritmo iterativo, y esto se debe a varias razones. Primero cuando el problema es sencillo y no requiere un gran gasto computacional, un algoritmo iterativo suele ser mejor que uno recursivo, como sucede en este caso. Ya que el metodo2 y el método3, con unas 1 y 2 llamadas recursivas respectivamente aumentan el tiempo de ejecución al crear una nueva entrada en la pila, guardar la dirección de retorno y volver de las propias llamadas puede aumentar el tiempo contundentemente.

Métodos Fibonacci

Metodo1 - iterativo:

- Complejidad: $O(n)$

Metodo2 - iterativo:

- Complejidad: $O(n)$

Metodo3 – por sustracción:

- Complejidad: $O(n)$

Metodo4 – por sustracción:

- Complejidad: $O(1,6^n)$

n	Metodo1	Metodo2	Metodo3	Método 4
40	237	493	643	4338

Aquí podemos notar una uniformidad en los 3 primeros métodos, que, aunque sigue ganando la parte iterativa, la recursiva no se aleja tanto del caso anterior, debido a que la secuencia Fibonacci con un n grande es un tema complejo de tratar. El método 4 es intratable en cuanto a tiempo por ser de tipo exponencial debido a que hace 2 llamadas recursivas con diferente resto, haciendo las llamadas mucho más difíciles de tratar.

Algoritmo Trivial

Tiempos con 1 repetición

N	Tiempos(ms)
1024	17
2048	16
4096	30
8192	200
16384	427
32768	1680
65536	6838
131072	28950
262144	163918

Algoritmo DyV repetición =1

N	Tiempos(ms)
1024	29
2048	20
4096	54
8192	199
16384	248
32768	950
65536	3679
131072	10016
262144	77357

Finalmente, después de los resultados anteriores, que el método recursivo es mucho más rápido que el método iterativo, ya que el primero trabaja con una complejidad $O(n^2)$ mientras que el divide y vencerás en su peor caso, el cual es que tenga que calcular varias veces la distancia mínima, es de esa complejidad. También la diferencia de tiempos se debe al coste del problema que mediante un algoritmo recursivo se resuelve mejor que uno iterativo.

Practica 3

La complejidad del algoritmo voraz realizado tanto en java como en Python es de complejidad $O(n^2)$, debido a que se realizan 2 bucles uno para recorrer todos los nodos del grafo y el segundo que recorre los vecinos, para ir determinando cuales son los colores disponibles, la toma de tiempos se realizó con el algoritmo en java con optimización y con 100 repeticiones debido a que con una repetición no se apreciaba la tendencia en tiempos del mismo.

JAVA

Repeticiones:100

n	t(ms)
8	7
16	3
32	4
64	8
128	11
256	21
512	41
1024	82
2048	190
4096	298
8192	631
16384	2174
32768	3647
65536	8287

Después de la medición queda claro el comportamiento en tiempos del algoritmo, creciendo cuadráticamente para unas 100 repeticiones, donde a medida que se aumenta el tamaño del programa o de la cantidad de nodos, mejor dicho, ya la cantidad de sus vecinos, la medición se dispara más rápido que el de un algoritmo lineal, pero menor que uno de tercer grado.

Practica 4 Programación dinámica

Distribución del ferry:

1. Definición del Estado: Los índices de la tabla usada para realizar el ejercicio, tienen el siguiente significado,

- Las filas o la “i” consiste en el número de vehículos introducidos en babor
- Las columnas o la “j” consiste en los metros de babor ocupados hasta el momento con los i vehículos

2. Relación de recurrencia:

- El caso base consiste en poner la fila 0, con false, excepto el primer elemento, esto para decir de alguna forma que meter 0 coches es posible en babor, y así para cuando se comprueba si la fila es completa de falsos no termine en la primera.
- Para determinar si un coche puede entrar en babor, se debe de comprobar que, si sumamos a la longitud de los vehículos que ya tenemos, la longitud del nuevo, y no superamos la longitud máxima en babor, podemos colocarlo allí.
Se expresa de la siguiente forma:

```
if(j+vehicles.get(i-1) <= boatlenght) {  
    matrix[i][j+vehicles.get(i-1)]=true;  
}
```

- Para determinar si un coche puede entrar en estribor, lo que hacemos es comprobar la suma acumulada de la longitud de los coches, suponiendo que los primeros i vehículos los hemos puesto allí, y eso le restamos la longitud del vehículo a colocar, si arroja un resultado menor que la medida de estribor, se coloca allí. En forma de código:

```
if(sumLenghtV[i] - j <= boatlenght) {  
    matrix[i][j]=true;  
}
```

3. Complejidad del Algoritmo: La complejidad del algoritmo depende tanto de la cantidad de vehículos, como de la longitud del barco, es decir, $O(n \cdot l)$, donde normalmente la complejidad esperada en casos normales sea cuadrática.

Practica 5: BackTracking

Santiago Araque

El algoritmo usado para determinar la cantidad de contenedores para almacenar una serie de números o pesos, en un tamaño específico, posee una complejidad cercana a la factorial, debido al incremento de opciones por elemento en cada iteración, es decir:

- Al inicio tienes un elemento, al cual tienes una única opción de manejo: crear un contenedor y guardarlo allí
- Después un segundo elemento, aumenta en uno las opciones, ya que se puede introducir en un contenedor ya creado por el primer elemento, o guardarlo en un nuevo
- Finalmente, en el tercer elemento, vuelve a aumentarse las elecciones, ya que se puede introducir en el primer, segundo o un nuevo contenedor.

Esto hace que las opciones y árbol aumente su tamaño en base a la cantidad de opciones a analizar, haciendo que la complejidad se dispare a cercana a factorial, pero con la poda introducida se puede disminuir el tiempo en casos donde se encuentra una solución óptima relativamente rápido, generando muchas podas y disminuyendo el tiempo considerablemente, pero no en todos los casos sucede así, tal como se ve en la siguiente tabla, donde cada test se repitió un total de 100 veces

Test	Tiempo (ms)
0	4
1	2
2	2
3	1
4	2
5	2
6	1
7	2
8	7302
9	FDT

Practica 6. Extra: BackTracking extra

Santiago Araque

El algoritmo a implementar fue el de las NReinas, que consiste en encontrar una combinación en un tablero determinado, la cantidad máxima de reinas que se pueden colocar sin que se puedan atacar unas a otras.

El problema se resolvió mediante BackTracking donde primero se introduce una reina, luego se comprueba si hay alguna reina en la columna, fila, o diagonal de la misma, si no hay alguna se coloca allí permanente hasta evaluar una nueva solución en el BackTracking, y así con las demás hasta llegar a colocar las n reinas correspondientes.

El comportamiento de este algoritmo es un poco diferente al anterior , ya que aquí no se crean nuevas opciones posibles para introducir un elemento , mientras se evalúa el objeto, es decir no hay un caso nuevo por cada objeto a introducir, ya que el tablero siempre es el mismo , sin embargo aquí sucede al contrario , por cada reina que coloques se reducen las posibilidades en 1 , es decir que cuando colocas una reina en un tablero en la primera columna , ya no puedes colocar otra en esa misma columna , y así sucesivamente, haciendo que la complejidad se dispare como máximo a $O(n!)$

Los tiempos del algoritmo corresponden a lo mencionado anteriormente, sin embargo, sucede un problema, la memoria o heap asignado a la ejecución del programa, ya que la lista para guardar soluciones para un tablero muy extenso puede albergar millones de posibles soluciones, ocurriendo una excepción y finalizando el programa, tal como se ve en la siguiente tabla de tiempos,

Tablero(nxn)	Tiempos(ms)
2	0
3	11
4	11
5	11
6	13
7	13
8	14
9	16
10	20
11	68
12	161
13	768
14	3883
15	Heap exception

Practica 7: Ramifica y poda

Para la realización de esta práctica se creó un heurístico capaz de rebajar el tiempo de ejecución y que pueda ejecutar correctamente el test 09, que antes sin hacer algunas modificaciones no era capaz de ejecutarlo correctamente.

Para ello se implementó el heurístico de la siguiente manera:

- Se creó una nueva variable local dentro de la función BackTracking, que funcionara como límite inferior, es decir el mínimo teórico de contenedores adicionales necesarios, y
- Su cálculo consiste en la suma de los elementos restantes por colocar más la capacidad de cada contenedor menos 1 (este es para que el resultado salga truncado), todo esto a su vez dividido por la capacidad, de tal forma que se aproxime los contenedores que como mínimo son necesarios para almacenar los elementos en el estado actual
- Así que cuando se tiene este número y se le suma el número de contenedores que ya hay completos o creados, y su resultado empeora el mejor k, cortamos esa rama.

Ahora con este pequeño y sustancial cambio, podemos reducir el tiempo del test 8 y del test 9, tal como veremos en la siguiente tabla, donde cada test se ejecutó un total de 100 veces:

Test	Tiempo (ms)
0	4
1	1
2	1
3	3
4	1
5	4
6	1
7	1
8	5
9	6

Es evidente el cambio para los últimos 2 test, que reducen gigantescamente su tiempo, al cortar más ramas de forma eficiente, haciendo que el algoritmo sea capaz de resolver problemas de mayor tamaño a menor tiempo.